UNIVERSITÀ DI PISA

Computer Engineering

Foundations of Cybersecurity

Implementation of a Secure Digital Signature Service

Student:

Klaudio Çaça

Academic Year 2024/2025

Sommar

1 INTRODUCTION

1.1 Project overview

This project focuses on the design and implementation of a **Digital Signature Service (DSS)** using C++ and the OpenSSL library. A DSS is a trusted third-party system that manages cryptographic keys and performs digital signing operations on behalf of users.

In this implementation, the DSS is responsible for creating, storing, and deleting public-private key pairs for authenticated users. Additionally, the service can generate digital signatures on user-supplied documents and provide public keys to verify those signatures.

Users are registered offline and receive the DSS's public key and a temporary password, which must be changed at the first login. Once authenticated, users can securely interact with the DSS via an encrypted channel that guarantees **Perfect Forward Secrecy**, **message integrity**, **non-replay**, and **non-malleability**.

The DSS supports the following operations:

- **ChangePassword**: Change the password of the user.
- **CreateKeys**: Generates and stores a key pair for the user.
- **GetPublicKey**: Returns the public key associated with a given user.
- **DeleteKeys**: Removes a user's key pair from the system.
- **SignDoc**: Produces a digital signature on a document on behalf of the user.
- **VerifyDoc**: Verify if a signed document belongs to that user using his public key.

All private keys are encrypted before being stored on the server. Secure communication and mutual authentication are enforced using a custom authentication protocol built over OpenSSL primitives.

This report outlines the design, security considerations, message formats, and protocol sequences that define the DSS architecture.

1.2 Project implementation (basic concept)

The project is structured around two main components:

- server.cpp
- client.cpp

These files implement the interaction between the Digital Signature Service (DSS) and the users' requesting operations such as key generation, signing, or public key retrieval. The server is designed to handle multiple clients concurrently through multithreading, allowing

simultaneous access to the DSS by different users. Each file is responsible for managing its respective side of the communication: server.cpp manages the core logic and processing of requests on the DSS side, while client.cpp enables users to securely connect to the service and invoke supported operations.

In addition to the main components, the project includes two protocol-specific files:

- protocol_server.cpp
- protocol_client.cpp

These modules implement the handshake and secure channel establishment logic between the client and the DSS. Before any operation can be executed, a secure communication channel must be set up using cryptographic primitives such as **Diffie-Hellman key exchange**, **initialization vectors (IVs)**, **nonces**, **hmacs** and other secure elements. These values are exchanged between the client and server to derive shared session keys that ensure **confidentiality**, **integrity**, and **perfect forward secrecy**.

Furthermore, these files handle mutual authentication. Once the secure channel is established, the client (e.g., Alice) authenticates to the DSS using a username and password. Likewise, the DSS is authenticated by the client by verifying the server public key (server_pub.pem).

All authentication exchanges are performed over the secured channel, ensuring protection against eavesdropping and tampering.

The project also includes two additional support modules:

- utility.cpp
- userdb.cpp

The utility.cpp file encapsulates all core cryptographic functionalities used throughout the system. This includes the generation of **Diffie-Hellman key pairs**, computation and verification of **digital signatures**, **HMAC** generation, and other low-level cryptographic operations required for securing communication and data integrity.

The userdb.cpp module manages all operations related to the user database. It handles tasks such as updating a user's password, storing and retrieving public-private key pairs, and accessing user information. All user data is persistently stored in a local file named users.db, which acts as the database backend for the DSS.

2. PROTOCOL DESIGN

2.1 Low level format of all the exchanged messages

2.1.1 Custom implementation of exchange messages

To ensure correct and reliable transmission of messages over the communication layer, each message is prefixed with its length. This approach allows the receiver to know exactly how many bytes to expect, avoiding issues with partial reads or message boundaries.

Each message exchanged between the client and server follows the same structure:

1. A 4-byte header indicating the length of the message.
2. The message payload itself.

To transmit a message, the length of the payload is first converted from host byte order to network byte order using the **htonl** function. This step ensures compatibility across different machine architectures, as byte ordering may vary between systems (little-endian vs big-endian). The converted length is sent first, followed by the raw message data.

Similarly, on the receiving side, the message length is first read and then converted back from network to host byte order using **ntohl**. This decoded length is then used to allocate the appropriate buffer size and receive the full payload.

Here is the code implementing this logic:

```
bool write_msg(int sock,const byte_vec& data){    //byte_vec is a <vector>
    //converts
    uint32_t len_net = htonl(data.size());
    //send msg length firts
    if (!writeFull(sock, &len_net, sizeof(len_net))) return false;

    //then send the msg data
    return writeFull(sock, data.data(), data.size());
}
bool read_msg(int sock, byte_vec& out){
    // Riceive the lenght msg first
    uint32_t net_len;
    if (!readFull(sock, &net_len, sizeof(net_len))) return false;

    //converts
    uint32_t len = ntohl(net_len);
    if(len <= 0) return false;
    out.resize(len);

    // Riceive the msg data
    return readFull(sock, out.data(), len);
}
```

2.1.2 Custom implementation of send() and recv() functions

All messages are exchanged using the standard send() and recv() primitives provided by the socket API. However, these functions do not guarantee that the entire buffer will be transmitted or received in a single call—they may return after processing only a portion of the data.

To address this limitation and ensure reliable transmission, custom wrapper functions are implemented to repeatedly invoke send() and recv() until the entire message buffer has been successfully sent or received. These utility functions guarantee that the full contents of each message are delivered before proceeding with further processing.

Function **readFull(..)** to receive repeatedly all the data in the buffer until it's empty:

```
static bool readFull(int sock, void* buffer, ssize_t length){
    char* ptr = static_cast<char*>(buffer);
    while(length > 0){
        ssize_t received = recv(sock, ptr, length, 0);
        if(received <= 0) return false;
        ptr += received;
        length -= received;
    }
    return true;
}
```

Function **writeFull(..)** to send repeatedly all the data in the buffer until it's empty:

```
static bool writeFull(int sock, const void* buffer, ssize_t length){

    const char* ptr = static_cast<const char*>(buffer);
    while (length > 0) {
        ssize_t sent = send(sock, ptr, length, 0);
        if (sent <= 0) return false;
        ptr += sent;
        length -= sent;
    }
    return true;
}
```

To summarize the **low level** message exchange mechanism: when sending a message, the length of the message is first converted to network byte order using htonl to ensure portability across different system architectures. This length is then sent as a 4-byte header, then the actual message payload is sent. Both the length and the payload are transmitted using the custom **writeFull** function, which ensures that all data is reliably sent by repeatedly calling the send() primitive until the entire buffer is transmitted.

The receiving process follows the same logic in reverse. First, the 4-byte length field is read using **readFull**, then converted back to host byte order using ntohl. The receiver then allocates the appropriate buffer and reads the full message payload using again readFull.

This approach guarantees that the client and server always know the exact size of the incoming message and can reliably exchange data over the communication channel.

2.2 Handshake between the client and the server

As I described in the introduction chapter users interact with the DSS through a secure channel that must be established before issuing every operation.

In order to ensure integrity and perfect forward secrecy in the channel an handshake function between the client and the user is done. Below is reported the flow analysis of the handshake:

1. Client generates a Diffie-Hellman key pair and extracts the public key. Then sends the client public key to the server using the custom write_msg function.

2. Server generates his Diffie-Hellman public key and a server nonce (s_nonce). Receives the client Diffie-Hellman public key using the custom read_msg function. The server then generates the **shared key** and from this it derives 4 main keys: **s_key**, **c_key**, **s_mac**, **c_mac** (**Perfect Forward Secrecy**).
3. The server inserts in a message iv + s_nonce + server public key (Diffie-Hellman) (length + data) + encrypted signature (using s_privkey.pem + iv to encrypt) + generate hmac (using s_mac).
   Finally sends the complete message to client.

4. Client receives the message and separates all the components. Generates the **shared key** using the server Diffie-Hellman public key and derives the four main keys: s_key, c_key, s_mac, c_mac. Verify hmac using s_mac just derived (**Integrity** + **Non-malleability**). Decrypt signature using iv + s_key. Verify signature using s_pubkey.pem (**Authentication of server)**.

Both server and client now store all the information necessary to communicate in a secure channel. The next chapter reports how the secure channel is done and how to properly use it.

2.3 Application Level format of all the exchanged messages

Both the client and the server now share the same four principal keys: s_key, c_key, s_mac, c_mac (**Perfect Forward Secrecy)**. Every time they want to send a message they also generate nonces: s_nonce for the server and c_nonce for client (**No Replay Attack**), to ensure freshness of messages.

The secure channel is now valid and this is how it is used for every operation between the client and the DSS server.

Client:
sendSecure(server_socket, c_nonce, s_nonce, **c_key**, **c_mac**, cmd, data);

recvSecure(server_socket, c_nonce, s_nonce, **s_key**, **s_mac**, cmd, data);

Server:
sendSecure(client_socket, c_nonce, s_nonce, **s_key**, **s_mac**, cmd, data);

recvSecure(clien_socket, c_nonce, s_nonce, **c_key**, **c_mac**, cmd, data);

When the client sends a message it uses its keys (c_key and c_mac), the server receives the message using those same keys to verify the client. Symmetrically the server sends a message using its keys (s_key and s_mac) and the client receives the message with the same keys of the server (they know all the four keys thanks to the handshake function).

2.3.1 SendSecure Implementation Function

Every time a client requests an operation this schema is followed. For simplicity in this schema, it's the client to send an operation and the server to receive it (but it is valid in opposite direction):

Client:

- Generates a new c_nonce (random bytes).
- Insert in a message both the new c_nonce and the old s_nonce (this s_nonce was given by the server in the very first handshake function), then insert also the command of the operation and the data (len + payload).
- Generate iv and encrypt this message (iv + c_key).
- Compute hmac (using c_mac).
- Stores iv + encrypted message + hmac in a message and sends to the server (using write_msg custom implementation).

2.3.2 RecvSecure Implementation Function

The server follows this schema when receiving an operation (valid also for a client):

Server:

- Read all the message (using the read_msg custom implementation).
- Divides iv, encrypted message and hmac.
- Check hmac (using c_mac because the message was sent by the client). **Integrity + Non-malleability**.
- Decrypt message (iv + c_key).
- The server now checks if the s_nonce received by the client is the same as the one he already has. This is done to prevent **Replay-Attacks** (attacker uses same message multiple times) and it ensure freshness. If it's the same s_nonce, then the server stores the new c_nonce given by the client in order that the client will do the same check in the very next message exchange.
- Now the server can read the command and the data for the operations.

2.4 Authentication of user

Now that the secure channel is created the authentication of a specific user can start securely. The authentication is automatically started right after the handshake function between the client and the server.

1. Client: Prompt input the username and the password of an user (e.g Alice). Insert in the data field the username + password + LOG command. Then SendSecure the data (secure channel).

2. Server: Receive LOG command and the username + password using RecvSecure. Verify if credentials meet in the database, if not sendSecure an ERR command, otherwise it sends:
    - Change (CHG) command if it is the first login of the user.
    - Aknowledge (ACK) command if user exists and it's not his first login.

3. Client: RecvSecure the command and reads the command:
    - ACK the user is logged in and can request all the operations to the DSS.
    - CHG it means the user must change the default password since it is his first login. Prompts new password and SendSecure to the server.
    - ERR user does not exists or password incorrect. Authentication is denied.

4. Server: (Only if CHG command was sent) RecvSecure the new password of the user. Updates the new password in the database.

Now if the client is authenticated correctly, he can perform all the possible operations handled by the DSS using the secure channel (sendSecure, recvSecure) as similarly as was done for the authentication operation.

3. COMPLETE FLOW ANALYSIS OF THE PROJECT

All the error operations are not described. If an error occur it's likely that the connection ends.

Connection phase:

Client

- correctly load server_pub.pem
- declare all the necessary variables
-connect to the server

Server

- correctly load server_priv.pem
- correctly load or reset database (users.db)
-accept a client connection

Handshake phase:

1. Client
- Generate Diffie-Hellman key pair
- Extract client public key
- Send his public key to the server
(using custom send function)

2. Server
- Receive client public key
- Generate Diffie-Hellman key pair
- Extract server public key

3. Server
- Generate shared key (client_pub_key + server_pub key)
- Derive 4 keys (s_key, c_key, s_mac, c_mac)

4. Server
- Generate iv and s_nonce
- generate signature (iv + server_priv.pem)
- encrypt signature (iv + s_key)
- compute hmac (s_mac)
- insert in a msg iv + server_pubkey + encrypted signature + hmac in a message
- Sends the message to the client

5. Client
- Receive full msg
- Divides iv + s_nonce + s_pubkey (dh) + encrypted signature + hmac
- Generates shared key with the s_pubkey (dh)
- Derives s_key, c_key, c_mac, s_mac
- Check hmac (s_mac)
- Decrypt signature (iv + s_key)
- Verify signature (server_pub.pem)

Authentication phase:

1. Client
- Prompt username + password (e.g. alice)
- SendSecure all keys + data + LOG command

2. Server
- Receives LOG + username + password
- Verify credentials in the database
- Sends ACK/ERR/CHG command (first login)

3. Client
- Receive ACK/ERR/CHG command
- If CHG prompts the new password
- Sends CHG + new password to server

4. Server
- Receives CHG + new password
- Updates new password in database

Request phase: similar way as authentication.

4. CONCLUSION

The implementation of this Digital Signature Service (DSS) demonstrates a secure, robust, and modular architecture for managing cryptographic keys and digital signatures within an organization. Through the use of C++ and OpenSSL, the system ensures confidentiality, integrity, authentication, and perfect forward secrecy via a custom-designed handshake protocol and secure communication layer.

The DSS allows users to authenticate, manage their key pairs, and request digital signing operations in a safe and efficient manner. Each phase—connection, handshake, authentication, and command execution—was carefully designed to meet strong security guarantees using well-established cryptographic primitives such as RSA, Diffie-Hellman, AES, and HMAC.

By modularizing the system into separate components (e.g., protocol handling, cryptographic utilities, user database), the project remains extensible and maintainable. The result is a complete end-to-end system capable of handling multiple clients simultaneously, securely exchanging messages, and enforcing strict authentication and authorization policies.

5. HOW TO START DSS

5.1 Generation of server keys (for authentication of server)

Before launching the DSS there need to be generated the server private and public key for the authentication of the server called ***server_priv.pem*** and ***server_pub.pem*** (they are different from the Diffie-Hellman key pairs that are freshly generated every time the server is executed).

Those two file are generated only once and from the terminal using the Openssl library. The key files must be generated in the same folder as the client and server executables.

To generate the private key:
openssl genpkey -algorithm RSA -out server_priv.pem -pkeyopt rsa_keygen_bits:2048

To generate the public key:
openssl pkey -in server_priv.pem -pubout -out server_pub.pem

Check correctness of the generated keys:

To check private key:
openssl pkey -in server_privkey.pem -text -noout

To check public key:
openssl pkey -pubin -in server_pub.pem -text -noout

5.2 Server and client compilation

Server: These files must be in the same folder in order to compile the server:

- server.cpp
- protocol_server.cpp + protocol_server.h
- utility.cpp + utility.h
- userdb.cpp + userdb.h
- types.h

Compile with this flag:
g++ -DSERVER server.cpp utility.cpp protocol_server.cpp userdb.cpp -o server -pthread -lssl -lcrypto

Client: these are the necessary files:

- client.cpp
- protocol_client.cpp + protocol_client.h
- utility.cpp + utility.h
- types.h

Compile with this flag:
g++ -DCLIENT client.cpp utility.cpp protocol_client.cpp -o client -pthread -lssl -lcrypto

For simplicity you can put the files in the same folder and compile like I described. Or create a makefile.

5.3 Server and client start

To start the project, we must have the executables client and server in the same folder as the server_priv.pem and server.pub.pem.

Server start:
./server reset

Adding the command reset forces the server to create the file users.db which has already initialized three users and their initial passwords (username, password):

Username	password
alice	alice
bob	bob
carl	carl

./server
Simply loads the users.db file if exists. It contains the last update for the information of the users.

Client start:

./client
Simply start a client (we can start multiple clients).

The client then must insert as input the username and password of the user he wants to access. If it securely authenticates this are the possible commands:

CHG to change password
CRT to create new pair of keys
DEL to delete pair of keys
GET to retrieve public key of a user
PBK to print the pubkey requested before
SGN to sign a specific document
VRF to verify a specific document
PRT to print all info (//debug on server)